



KOHAT UNIVERSITY OF SCIENCE & TECHNOLOGY
INSTITUTE OF COMPUTING

CS111 Object Oriented Programming & Design

Course Incharge Muhammad Ali Zeb

Program: BSSE/BSCS
Marks: 50

Final-Term Exam
Date: July 01, 2024

Semester: Spring-2024
Time Allowed: 2 hours

Attempt all questions. Explain your code with comments.

Question No.1) Construct a UML class diagram that shows the structure of a Course Management System having classes: Course, Library, Book, Person, Student, and Staff. A person can only be a Student or a Staff. Include the relationships, instance variables and methods to enable the names, addresses, and courses of students and staff, the name, phone number, and books in the Library, course code, course name and associated book for the Course, and book title for the Book, wherein students pay an annual fee, each course has a registration fee, the library has a membership fee, books have a purchase price, and staff members are paid a salary. (10 marks)

Question No.2) Implement a **Library** class. A library object keeps track of the following information: library's name, library's phone number, and a collection of books. A book has attributes book id, book title, author name. All the instance variables should be private. (2 marks)

- Write an **addBook** method that receives a **Book** object and adds it to the collection if the library does not contain that book. (2 marks)
- Write a **deleteBook** method that receives a **Book** object and removes it from the collection after checking if the library contains that book. (2 marks)
- Write a **toString()** method that prints the details of the library. The method should also print the number of books in the library. (2 marks)
- Write a **Test** class with a **main** method that creates two **Book** objects and a **Library** object after adding the book to the library. Print the library object using **System.out.println()**. (2 marks)

Question No.3) Write code to store information for at least three different species of animals. The information of interest is the sound, name and color of the animal. Each class must implement a common function for the movement of each type of animal, that prints that the animal has moved. Each animal has its own type of movement (birds fly, cats walk, snakes crawl etc). Write a test class that polymorphically displays the features of each animal (use **toString()**), and the way the animal has moved. What changes will you make if you want to have the option to initialize the objects for the classes without the initial values for the instance variables. (10 marks)

Question No.4) How would you use interfaces if you were to add the same movement functionality in Question 3 to a different class, **Vehicle**, and its subclasses (**Car**, **Bus**, **Bike**), with attributes **model**, **color**, **price**. The **Vehicle** subclasses must also implement a common method that returns the price for each object of the **Vehicle** class. Write code for only one subclass only. Polymorphically display the movement for all the objects having the movement functionality, and the price for all the **Vehicle** objects. If an object is an instance of any of the **Vehicle** class (**Car**, **Bus**, **Bike**), it should display additional information of "Starting the Vehicle..." before displaying it has moved. (10 marks)

Question No.5) Explain each of the following Object-Oriented Programming Concepts. Use code statements to illustrate your answer.

- composition (2 marks)
- interface (2 marks)
- abstract classes and methods (2 marks)
- polymorphism (2 marks)
- access modifiers (2 marks)

Question 1: UML Class Diagram for Course Management System*

The class structure includes:

- Course

- Library

- Book

- Person (Abstract) → Student, Staff

python

```
from abc import ABC, abstractmethod
```

```
class Book:
```

```
    def __init__(self, book_id, title, purchase_price):
```

```
        self.book_id = book_id
```

```
        self.title = title
```

```
        self.purchase_price = purchase_price
```

```
    def __str__(self):
```

```
        return f"Book ID: {self.book_id}, Title: {self.title}, Price: {self.purchase_price}"
```

```
class Library:
```

```
    def __init__(self, name, phone):
```

```
        self.name = name
```

```
        self.phone = phone
```

```
        self.books = []
```

```
    def add_book(self, book):
```

```
        if book not in self.books:
```

```
            self.books.append(book)
```

```
def remove_book(self, book):  
    self.books = [b for b in self.books if b.book_id != book.book_id]
```

```
def __str__(self):  
    return f"Library: {self.name}, Phone: {self.phone}, Books: {len(self.books)}"
```

```
class Course:  
    def __init__(self, course_code, course_name, registration_fee, associated_book):  
        self.course_code = course_code  
        self.course_name = course_name  
        self.registration_fee = registration_fee  
        self.associated_book = associated_book
```

```
class Person(ABC):  
    def __init__(self, name, phone):  
        self.name = name  
        self.phone = phone
```

```
@abstractmethod  
def get_details(self):  
    pass
```

```
class Student(Person):  
    def __init__(self, name, phone, annual_fee):  
        super().__init__(name, phone)  
        self.annual_fee = annual_fee  
  
    def get_details(self):  
        return f"Student: {self.name}, Phone: {self.phone}, Annual Fee: {self.annual_fee}"
```

```
class Staff(Person):

    def __init__(self, name, phone, salary):

        super().__init__(name, phone)

        self.salary = salary


    def get_details(self):

        return f"Staff: {self.name}, Phone: {self.phone}, Salary: {self.salary}"
```

Question 2: Library Class Implementation

python

```
class Library:

    def __init__(self, name, phone):

        self.name = name

        self.phone = phone

        self.books = []


    def add_book(self, book):

        if book not in self.books:

            self.books.append(book)


    def delete_book(self, book):

        self.books = [b for b in self.books if b.book_id != book.book_id]


    def __str__(self):

        return f"Library: {self.name}, Phone: {self.phone}, Books: {len(self.books)}"
```

```
# Test Library

book1 = Book(1, "Python Basics", 500)
book2 = Book(2, "OOP with Python", 700)
library = Library("City Library", "123456789")
```

```
library.add_book(book1)
library.add_book(book2)
print(library) # Display library details
```

```
---
```

Question 3: Animal Class Hierarchy

```
python
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name, color):
        self.name = name
        self.color = color

    @abstractmethod
    def move(self):
        pass

    def __str__(self):
        return f"{self.name}, Color: {self.color}"
```

```
class Bird(Animal):  
    def move(self):  
        print(f"{self.name} flies.")
```

```
class Cat(Animal):  
    def move(self):  
        print(f"{self.name} walks.")
```

```
class Snake(Animal):  
    def move(self):  
        print(f"{self.name} crawls.")
```

```
# Test Polymorphism
```

```
animals = [Bird("Parrot", "Green"), Cat("Tiger", "Orange"), Snake("Python", "Brown")]
```

```
for animal in animals:
```

```
    print(animal)
```

```
    animal.move()
```

Question 4: Vehicle Class with Movement and Price*

```
python
```

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):  
    def __init__(self, model, color, price):  
        self.model = model  
        self.color = color  
        self.price = price
```

```
@abstractmethod
```

```
def move(self):
```

```
    pass
```

```
def get_price(self):
```

```
    return self.price
```

```
def __str__(self):
```

```
    return f"Model: {self.model}, Color: {self.color}, Price: ${self.price}"
```

```
class Car(Vehicle):
```

```
    def move(self):
```

```
        print(f"Starting the Car... {self.model} is moving.")
```

```
class Bus(Vehicle):
```

```
    def move(self):
```

```
        print(f"Starting the Bus... {self.model} is moving.")
```

```
class Bike(Vehicle):
```

```
    def move(self):
```

```
        print(f"Starting the Bike... {self.model} is moving.")
```

```
# Test Vehicles
```

```
vehicles = [Car("Toyota", "Red", 20000), Bus("Volvo", "Blue", 50000), Bike("Yamaha", "Black", 10000)]
```

```
for vehicle in vehicles:
```

```
    print(vehicle)
```

```
    vehicle.move()
```

```
---
```

Question 5: Object-Oriented Programming Concepts

(a) Composition

python

```
class Engine:
```

```
    def start(self):
```

```
        print("Engine started.")
```

```
class Car:
```

```
    def __init__(self):
```

```
        self.engine = Engine()
```

```
    def start_car(self):
```

```
        self.engine.start()
```

```
car = Car()
```

```
car.start_car()
```

(b) Interface (Using ABC)

python

```
class Movable(ABC):
```

```
    @abstractmethod
```

```
    def move(self):
```

```
        pass
```

```
class Robot(Movable):
```

```
    def move(self):
```



```
print("Robot is moving.")
```

```
robot = Robot()
```

```
robot.move()
```

```
#### *(c) Abstract Classes and Methods*
```

```
python
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def draw(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def draw(self):
```

```
        print("Drawing a circle.")
```

```
shape = Circle()
```

```
shape.draw()
```

```
#### *(d) Polymorphism*
```

```
python
```

```
class Animal:
```

```
    def sound(self):
```

```
        print("Animal makes a sound.")
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
print("Dog barks.")
```

```
animal = Dog()
```

```
animal.sound()
```

```
#### *(e) Access Modifiers*
```

```
python
```

```
class Example:
```

```
    def __init__(self):
```

```
        self.__privateVar = 10 # Private
```

```
        self._protectedVar = 20 # Protected
```

```
        self.publicVar = 30 # Public
```

```
obj = Example()
```

```
# print(obj.__privateVar) # Would raise an error
```

```
print(obj._protectedVar) # Accessible
```

```
print(obj.publicVar) # Accessible
```

